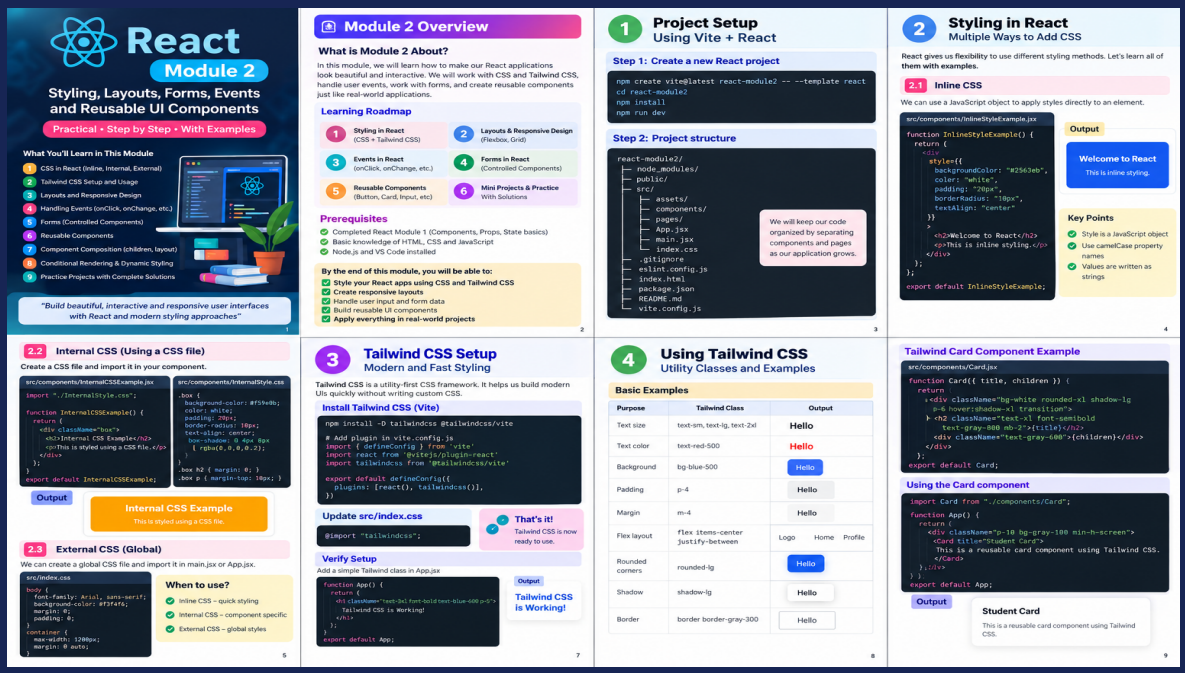
REACT

MODULE 2

Styling • Layouts • Events • Forms

Pure CSS + Tailwind CSS • Multiple cases • Visual diagrams • Step-by-step projects

Classroom-ready notes



AI-generated visual reference • UI concepts represented as learning diagrams

Module 2: What You Will Learn

Module 1 taught the basic React mental model. Module 2 turns those components into usable interfaces. We will style components, build responsive layouts, respond to events, work with forms, compose reusable UI, and compare ordinary CSS with Tailwind CSS.

Learning roadmap

1. **Styling in React** • Inline, component CSS, global CSS
2. **Tailwind CSS** • Current Vite setup and utility classes
3. **Layouts** • Flexbox, Grid and responsive design
4. **Events** • Click, input, submit, propagation
5. **Forms** • Controlled inputs and validation basics
6. **Reusable UI** • Buttons, cards, badges, composition
7. **Dynamic styling** • Classes based on props/data
8. **Projects** • Profile, dashboard and form projects

Prerequisites

Complete Module 1. Students should already understand JSX, components, props, map(), arrays and basic JavaScript functions.

By the end of Module 2

- Style React components using pure CSS and Tailwind CSS.
- Build responsive layouts with Flexbox and CSS Grid.
- Handle click, change, submit, keyboard and mouse events.
- Understand the difference between passing a function and calling a function.
- Build controlled form inputs.
- Create reusable Button, Card, Badge and Input components.
- Use props to control both appearance and behavior.
- Build a complete mini dashboard using the techniques in this module.

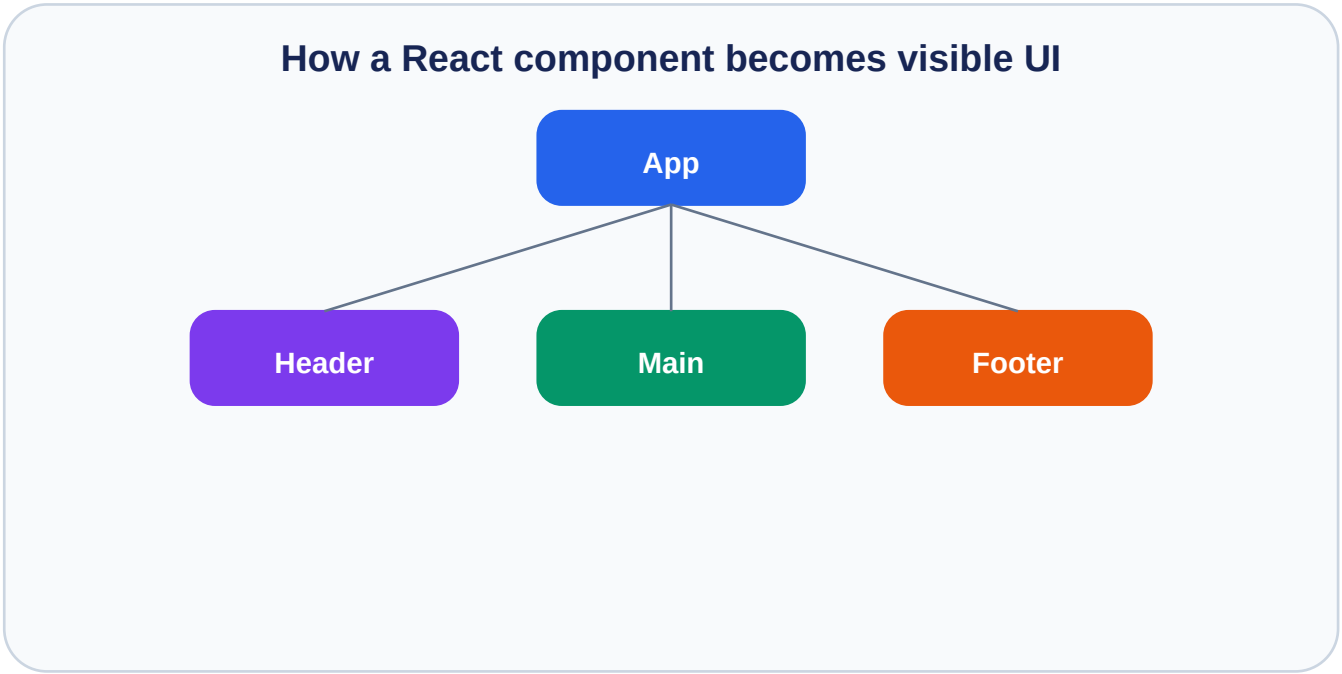
MODULE 2 • PART 1

Styling in React: Pure CSS from Simple to Reusable

1.1 Three common styling approaches

- **Inline styles:** useful for small dynamic values and quick experiments.
- **CSS files:** useful for reusable styles, pseudo-classes, media queries and larger styling systems.
- **Tailwind:** utility classes directly express many visual decisions in JSX.

Visual idea: component → markup → style → browser UI



1.2 Inline CSS

src/components/Welcome.jsx

```
function Welcome() {
  const boxStyle = {
    backgroundColor: "#2563eb",
    color: "white",
    padding: "24px",
    borderRadius: "12px",
    textAlign: "center"
  }

  return (
    <section style={boxStyle}>
      <h2>Welcome to React</h2>
      <p>This component uses inline styling.</p>
    </section>
  )
}

export default Welcome
```

Important inline-style rules

- CSS property names use camelCase: backgroundColor, borderRadius, fontSize.
- Values are normally strings unless the browser accepts a number directly.

• Inline styles do not naturally provide pseudo-classes such as :hover.

1.3 Internal/component CSS

Card.jsx

```
// src/components/Card.jsx
import "./Card.css"

function Card({ title, description }) {
  return (
    <article className="card">
      <h2 className="card__title">{title}</h2>
      <p className="card__description">{description}</p>
    </article>
  )
}

export default Card
```

Card.css

```
.card {
  padding: 20px;
  border: 1px solid #cbd5e1;
  border-radius: 16px;
  background: white;
  transition: transform 0.2s ease;
}

.card:hover {
  transform: translateY(-4px);
}

.card__title {
  margin: 0 0 8px;
  color: #172554;
}

.card__description {
  color: #64748b;
}
```

Why a CSS file is powerful

- Supports hover and focus states.
- Supports media queries.
- Keeps JSX cleaner.
- Makes repeated styles easier to maintain.
- Allows transitions, animations and more advanced selectors.

1.4 Global CSS

src/index.css

```
/* src/index.css */
* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: Arial, sans-serif;
  background: #f8fafc;
  color: #172033;
}

button,
input,
textarea,
select {
  font: inherit;
}
```

Importing component CSS vs global CSS

Imports

```
// main.jsx
import "./index.css"

// App.jsx
import "./App.css"

// Card.jsx
import "./Card.css"
```

Mental model

Global CSS establishes the page baseline. Component CSS handles a component's visual system. Inline styles are best reserved for small, truly dynamic values.

1.5 The same card in three styles

Case A: inline styling is fastest. **Case B:** CSS classes are clearer for reusable designs. **Case C:** Tailwind is concise when a utility-first system is desired.

MODULE 2 • PART 2

Tailwind CSS: Current Vite Setup

2.1 What Tailwind does

Tailwind provides utility classes such as p-4, rounded-xl, text-lg, bg-blue-600 and flex. Instead of inventing a class name for every visual property, you compose utilities in JSX. Tailwind's current Vite documentation uses the @tailwindcss/vite plugin and an @import "tailwindcss" in the CSS entry file.
■cite■turn0search1■turn0search10■

2.2 Current Tailwind + Vite setup

Terminal

```
npm create vite@latest react-tailwind-app -- --template react
cd react-tailwind-app
npm install
npm install tailwindcss @tailwindcss/vite
```

vite.config.js

```
// vite.config.js
import { defineConfig } from "vite"
import react from "@vitejs/plugin-react"
import tailwindcss from "@tailwindcss/vite"

export default defineConfig({
  plugins: [
    react(),
    tailwindcss()
  ]
})
```

src/index.css

```
/* src/index.css */
@import "tailwindcss";
```

App.jsx

```
// src/App.jsx
function App() {
  return (
    <h1 className="text-3xl font-bold text-blue-600">
      Tailwind is working!
    </h1>
  )
}

export default App
```

Start development server

```
npm run dev
```

Version warning

Older tutorials often show tailwind.config.js, @tailwind base, @tailwind components and @tailwind utilities. Those instructions are associated with Tailwind v3. Current Tailwind v4 uses the Vite plugin workflow shown above. ■cite■turn0search1■turn0search6■

2.3 Tailwind utility categories

Purpose	Example	Meaning
Text size	text-xl	Large text
Text color	text-red-500	Red text
Background	bg-blue-600	Blue background
Padding	p-4	Padding on all sides
Margin	mt-4	Top margin
Rounded	rounded-xl	Rounded corners
Shadow	shadow-lg	Large shadow
Flex	flex items-center	Flex layout + vertical centering
Grid	grid grid-cols-3	Three-column grid

2.4 Tailwind card

Reusable Tailwind Card

```
function Card({ title, children }) {
  return (
    <article className="
      rounded-xl
      border border-slate-200
      bg-white
      p-6
      shadow-lg
    ">
      <h2 className="mb-2 text-xl font-bold text-slate-800">
        {title}
      </h2>

      <div className="text-slate-600">
        {children}
      </div>
    </article>
  )
}

export default Card
```

2.5 Same UI: Pure CSS vs Tailwind

React + pure CSS

```
// Pure CSS
function ProfileCard({ name, role }) {
  return (
    <article className="profile-card">
      <h2 className="profile-card__name">{name}</h2>
      <p className="profile-card__role">{role}</p>
    </article>
  )
}
```

ProfileCard.css

```
.profile-card {
  padding: 24px;
  border-radius: 16px;
  background: white;
  border: 1px solid #e2e8f0;
  box-shadow: 0 10px 30px rgba(15,23,42,.08);
}

.profile-card__name {
  margin: 0 0 8px;
  font-size: 1.25rem;
  font-weight: 700;
}

.profile-card__role {
  margin: 0;
  color: #64748b;
}
```

React + Tailwind

```
// Tailwind
function ProfileCard({ name, role }) {
  return (
    <article className="
      rounded-2xl border border-slate-200
      bg-white p-6 shadow-xl
    ">
      <h2 className="mb-2 text-xl font-bold">
        {name}
      </h2>
      <p className="text-slate-500">{role}</p>
    </article>
  )
}
```

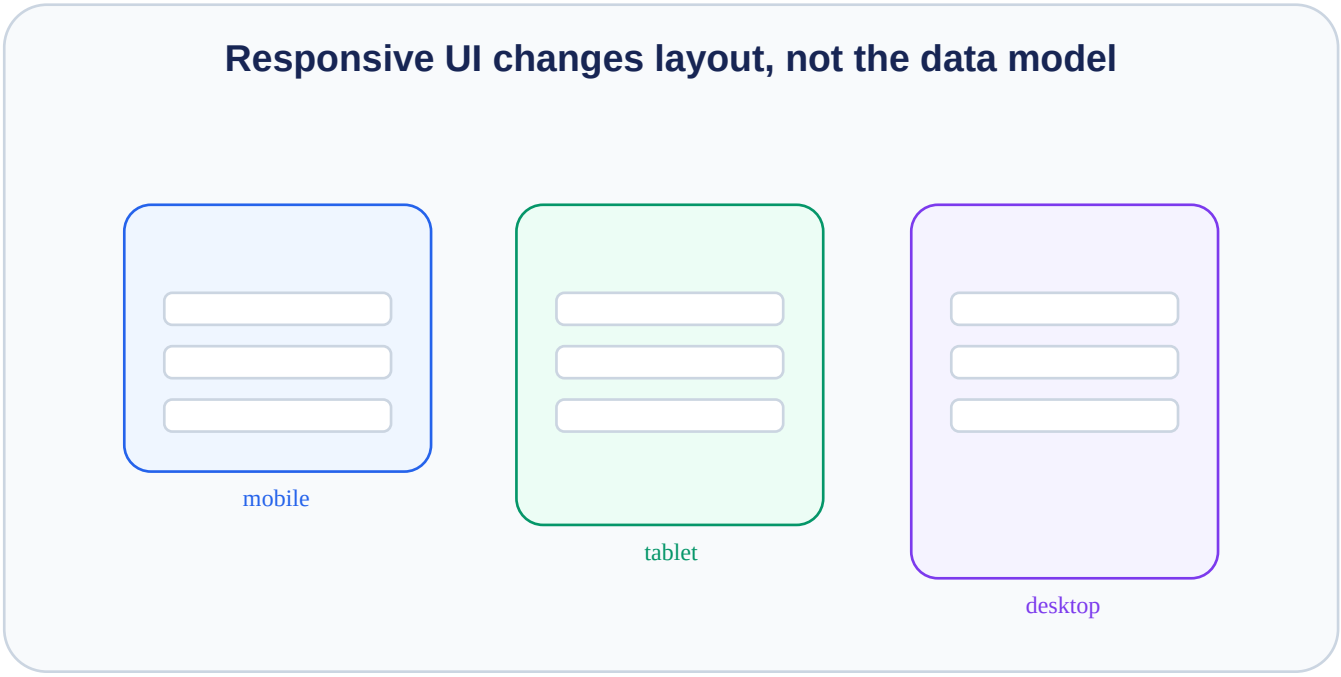
Teaching comparison

Pure CSS teaches the underlying styling system. Tailwind teaches composition through utilities. Students should be able to read both even if they later prefer one.

MODULE 2 • PART 3

Layouts: Flexbox, Grid & Responsive Design

3.1 Flexbox mental model



Flexbox is ideal when you are arranging items primarily along one dimension: a row or a column.

Navbar.jsx

```
function Navbar() {
  return (
    <nav className="navbar">
      <h1>My App</h1>

      <div className="navbar__links">
        <a href="#">Home</a>
        <a href="#">Courses</a>
        <a href="#">Contact</a>
      </div>
    </nav>
  )
}
```

Pure CSS Flexbox

```
.navbar {
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 16px 24px;
}

.navbar__links {
  display: flex;
  gap: 20px;
}
```

Tailwind Flexbox

```
<nav className="flex items-center justify-between p-4">
  <h1 className="text-xl font-bold">My App</h1>

  <div className="flex gap-5">
    <a href="#">Home</a>
    <a href="#">Courses</a>
    <a href="#">Contact</a>
  </div>
</nav>
```

3.2 CSS Grid for cards

Pure CSS responsive grid

```
.course-grid {
  display: grid;
  grid-template-columns:
    repeat(auto-fit, minmax(220px, 1fr));
  gap: 20px;
}
```

Tailwind arbitrary grid value

```
<div className="
  grid
  grid-cols-[repeat(auto-fit,minmax(220px,1fr))]
  gap-5
">
  <CourseCard />
  <CourseCard />
  <CourseCard />
</div>
```

3.3 Responsive breakpoints

Tailwind responsive layout

```
<div className="
  grid
  grid-cols-1
  sm:grid-cols-2
  lg:grid-cols-4
  gap-4
">
  ...
</div>
```

Read it as: mobile first → one column; small screens → two columns; large screens → four columns.

Pure CSS media queries

```
/* Pure CSS */
.course-grid {
  display: grid;
  grid-template-columns: 1fr;
}

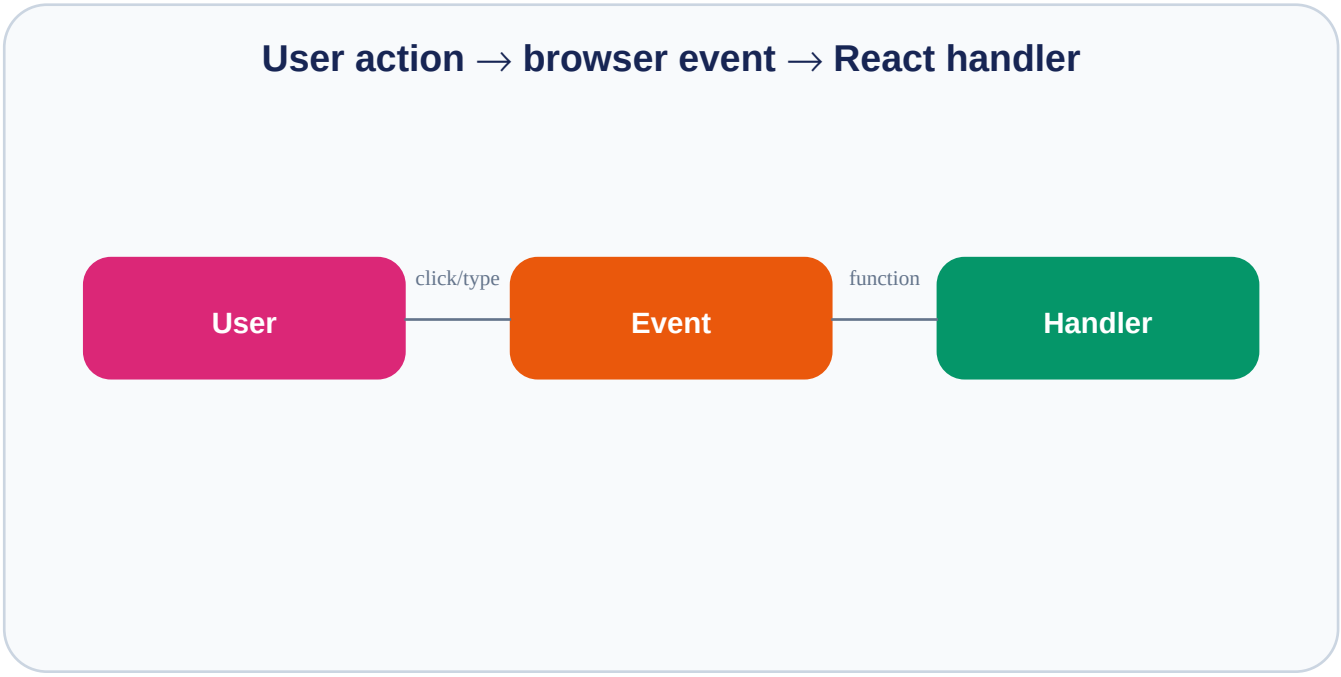
@media (min-width: 640px) {
  .course-grid {
    grid-template-columns: repeat(2, 1fr);
  }
}

@media (min-width: 1024px) {
  .course-grid {
    grid-template-columns: repeat(4, 1fr);
  }
}
```

MODULE 2 • PART 4

Events: Clicks, Inputs, Submit & Event Flow

4.1 React event model



React lets you pass event-handler functions to JSX elements. The official React documentation emphasizes passing the function rather than calling it: onClick={handleClick}, not onClick={handleClick()}. ■cite■turn0search0■turn0search2■

Correct event handler

```
function Button() {
  function handleClick() {
    alert("Button clicked!")
  }

  return (
    <button onClick={handleClick}>
      Click me
    </button>
  )
}

export default Button
```

Three cases

```
// ■ Calls immediately during rendering
<button onClick={handleClick()}>
  Click me
</button>

// ■ Passes the function
<button onClick={handleClick}>
  Click me
</button>

// ■ Inline function
<button onClick={() => alert("Hello")}>
  Click me
</button>
```

4.2 onClick with props

Parent supplies behavior

```
function Button({ onClick, children }) {
  return (
    <button onClick={onClick}>
      {children}
    </button>
  )
}

function App() {
  function handleSave() {
    alert("Saved!")
  }

  return (
    <Button onClick={handleSave}>
      Save
    </Button>
  )
}
```

Reusable component pattern

A Button component can own appearance while the parent owns what should happen when the button is clicked. This separation becomes extremely useful in larger applications.

4.3 Mouse and keyboard events

Multiple event types

```
function InteractiveBox() {
  function handleMouseEnter() {
    console.log("Mouse entered")
  }

  function handleKeyDown(event) {
    console.log("Key:", event.key)
  }

  return (
    <div
      tabIndex="0"
      onMouseEnter={handleMouseEnter}
      onKeyDown={handleKeyDown}
    >
      Hover or focus me
    </div>
  )
}
```

4.4 Event object

Reading input value

```
function SearchBox() {
  function handleChange(event) {
    console.log(event.target.value)
  }

  return (
    <input
      type="text"
      onChange={handleChange}
      placeholder="Search..."
    />
  )
}
```

4.5 preventDefault()

Prevent page reload

```
function SignupForm() {
  function handleSubmit(event) {
    event.preventDefault()
    console.log("Form submitted")
  }

  return (
    <form onSubmit={handleSubmit}>
      <input type="email" />
      <button type="submit">
        Sign up
      </button>
    </form>
  )
}
```

4.6 stopPropagation()

Stopping propagation

```
function Parent() {
  function handleParentClick() {
    console.log("Parent clicked")
  }

  function handleChildClick(event) {
    event.stopPropagation()
    console.log("Child clicked")
  }

  return (
    <div onClick={handleParentClick}>
      <button onClick={handleChildClick}>
        Click
      </button>
    </div>
  )
}
```

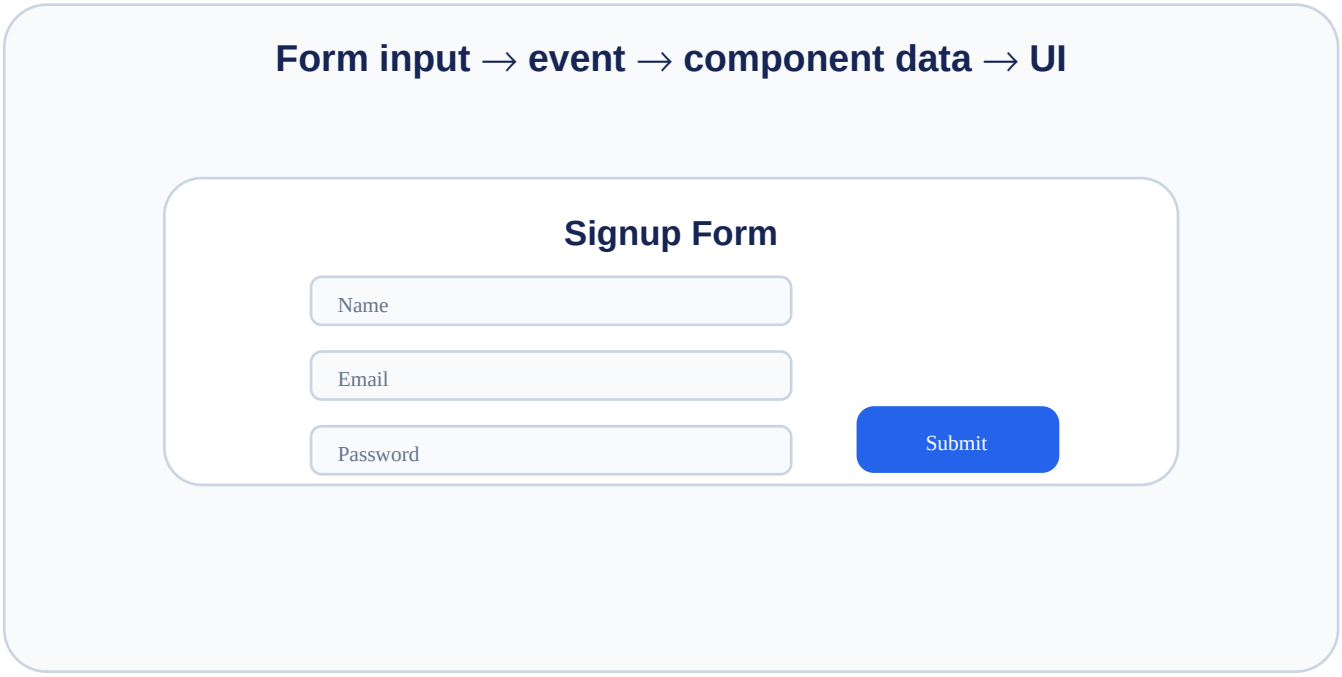
Do not confuse them

preventDefault() stops a browser's default behavior. stopPropagation() stops the event from continuing to parent handlers. They solve different problems. ■cite■turn0search0■

MODULE 2 • PART 5

Forms: Controlled Inputs & Practical Validation

5.1 Form mental model



In React, a controlled input gets its displayed value from component state. State is formally introduced in the next module, but forms are important enough that we will preview the pattern here.

Controlled input preview

```
import { useState } from "react"

function LoginForm() {
  const [email, setEmail] = useState("")

  function handleSubmit(event) {
    event.preventDefault()
    console.log(email)
  }

  return (
    <form onSubmit={handleSubmit}>
      <input
        type="email"
        value={email}
        onChange={(event) =>
          setEmail(event.target.value)
        }
      />

      <button type="submit">
        Login
      </button>
    </form>
  )
```

Controlled input preview (continued)

```
}

export default LoginForm
```

5.2 Multiple fields

Multiple controlled fields

```
import { useState } from "react"

function RegisterForm() {
  const [form, setForm] = useState({
    name: "",
    email: "",
    password: ""
  })

  function handleChange(event) {
    const { name, value } = event.target

    setForm({
      ...form,
      [name]: value
    })
  }

  function handleSubmit(event) {
    event.preventDefault()
    console.log(form)
  }

  return (
    <form onSubmit={handleSubmit}>
```

Multiple controlled fields (continued)

```
      <input
        name="name"
        value={form.name}
        onChange={handleChange}
      />

      <input
        name="email"
        value={form.email}
        onChange={handleChange}
      />

      <input
        name="password"
        type="password"
        value={form.password}
        onChange={handleChange}
      />

      <button type="submit">Create account</button>
    </form>
  )
}
```

5.3 Basic validation cases

Simple validation

```
function RegisterForm() {
  const [email, setEmail] = useState("")

  function handleSubmit(event) {
    event.preventDefault()

    if (!email) {
      alert("Email is required")
      return
    }

    if (!email.includes("@")) {
      alert("Enter a valid email")
      return
    }

    console.log("Valid form")
  }

  return (
    <form onSubmit={handleSubmit}>
      <input
        value={email}
        onChange={(event) =>
          setEmail(event.target.value)
```

Simple validation (continued)

```
        }
      />

      <button type="submit">
        Submit
      </button>
    </form>
  )
}
```

5.4 Select and checkbox

Select + checkbox

```
function Preferences() {
  const [role, setRole] = useState("student")
  const [accepted, setAccepted] = useState(false)

  return (
    <form>
      <select
        value={role}
        onChange={(event) =>
          setRole(event.target.value)
        }
      >
        <option value="student">Student</option>
        <option value="teacher">Teacher</option>
      </select>

      <label>
        <input
          type="checkbox"
          checked={accepted}
          onChange={(event) =>
            setAccepted(event.target.checked)
          }
        />
        Accept terms
```

Select + checkbox (continued)

```
      </label>
    </form>
  )
}
```

MODULE 2 • PART 6

Reusable UI Components: Button, Badge, Input & Card

6.1 Component design rule

A reusable component should expose the decisions that need to vary and keep the rest of its implementation internal. For example, a Button may receive children, variant, disabled and onClick.

Reusable Button

```
function Button({
  children,
  variant = "primary",
  disabled = false,
  onClick
}) {
  return (
    <button
      disabled={disabled}
      onClick={onClick}
      className={`btn btn--${variant}`}
    >
      {children}
    </button>
  )
}

export default Button
```

Button.css

```
.btn {
  border: 0;
  border-radius: 10px;
  padding: 10px 16px;
  cursor: pointer;
  font-weight: 700;
}

.btn--primary {
  background: #2563eb;
  color: white;
}

.btn--danger {
  background: #dc2626;
  color: white;
}

.btn--secondary {
  background: #e2e8f0;
  color: #172033;
}
```

6.2 Same Button with Tailwind

Tailwind Button

```
function Button({
  children,
  variant = "primary",
  disabled = false,
  onClick
}) {
  const styles = {
    primary:
      "bg-blue-600 text-white hover:bg-blue-700",
    danger:
      "bg-red-600 text-white hover:bg-red-700",
    secondary:
      "bg-slate-200 text-slate-900 hover:bg-slate-300"
  }

  return (
    <button
      disabled={disabled}
      onClick={onClick}
      className={`
        rounded-lg px-4 py-2
        font-bold transition
        disabled:cursor-not-allowed
        disabled:opacity-50
        ${styles[variant]}
```

Tailwind Button (continued)

```
      `}
    >
      {children}
    </button>
  )
}

export default Button
```

6.3 Badge component

Pure CSS Badge

```
function Badge({ children, type = "success" }) {
  const classes = {
    success: "badge badge--success",
    warning: "badge badge--warning",
    danger: "badge badge--danger"
  }

  return (
    <span className={classes[type]}>
      {children}
    </span>
  )
}

export default Badge
```

Badge.css

```
.badge {
  display: inline-block;
  padding: 4px 10px;
  border-radius: 999px;
  font-size: 0.8rem;
  font-weight: 700;
}

.badge--success {
  background: #dcfce7;
  color: #166534;
}

.badge--warning {
  background: #fef3c7;
  color: #92400e;
}

.badge--danger {
  background: #fee2e2;
  color: #991b1b;
}
```

6.4 Reusable Input

Input.jsx

```
function Input({
  label,
  name,
  type = "text",
  value,
  onChange,
  placeholder
}) {
  return (
    <label className="field">
      <span className="field__label">
        {label}
      </span>

      <input
        className="field__input"
        name={name}
        type={type}
        value={value}
        onChange={onChange}
        placeholder={placeholder}
      />
    </label>
  )
}
```

Input.jsx (continued)

```
export default Input
```

Input.css

```
.field {
  display: grid;
  gap: 6px;
}

.field__label {
  font-weight: 700;
}

.field__input {
  width: 100%;
  border: 1px solid #cbd5e1;
  border-radius: 10px;
  padding: 10px 12px;
}

.field__input:focus {
  outline: 3px solid #dbeafe;
  border-color: #2563eb;
}
```

Accessibility habit

Use a real for form fields and real elements for buttons. Styling should change appearance, not remove useful browser semantics. React's event guidance also recommends appropriate HTML elements for interactions. ■cite■turn0search0■

MODULE 2 • PART 7

Dynamic Styling & Real-World Cases

7.1 Status card with data-driven styling

Pure CSS dynamic class

```
function StatusCard({ title, status }) {
  const statusClass = {
    online: "status status--online",
    offline: "status status--offline",
    busy: "status status--busy"
  }

  return (
    <article className="card">
      <h2>{title}</h2>
      <span className={statusClass[status]}>
        {status}
      </span>
    </article>
  )
}
```

CSS

```
.status {
  display: inline-block;
  padding: 5px 10px;
  border-radius: 999px;
  font-weight: 700;
}

.status--online {
  background: #dcfce7;
  color: #166534;
}

.status--offline {
  background: #e2e8f0;
  color: #475569;
}

.status--busy {
  background: #fee2e2;
  color: #991b1b;
}
```

7.2 Tailwind dynamic style

Tailwind dynamic classes

```
function StatusCard({ title, status }) {
  const statusClass = {
    online:
      "bg-green-100 text-green-700",
    offline:
      "bg-slate-100 text-slate-700",
    busy:
      "bg-red-100 text-red-700"
  }

  return (
    <article className="rounded-xl bg-white p-5 shadow">
      <h2 className="mb-3 font-bold">{title}</h2>

      <span className={`
        rounded-full px-3 py-1
        text-sm font-bold
        ${statusClass[status]}
      `}>
        {status}
      </span>
    </article>
  )
}
```

7.3 Conditional class with ternary

Ternary class

```
function Alert({ type, message }) {
  return (
    <div
      className={
        type === "error"
          ? "alert alert--error"
          : "alert alert--success"
      }
    >
      {message}
    </div>
  )
}
```

7.4 Disabled UI

Disabled state styling

```
function SubmitButton({ loading }) {
  return (
    <button
      disabled={loading}
      className={`
        rounded-lg px-4 py-2
        font-bold
        ${loading
          ? "bg-slate-400 text-white"
          : "bg-blue-600 text-white"}
      `}
    >
      {loading ? "Submitting..." : "Submit"}
    </button>
  )
}
```

7.5 Active navigation link preview

Reusable active link

```
function NavLink({ active, children }) {
  return (
    <a
      href="#"
      className={`
        rounded-md px-3 py-2
        ${active
          ? "bg-blue-600 text-white"
          : "text-slate-600 hover:bg-slate-100"}
      `}
    >
      {children}
    </a>
  )
}
```

Pattern to remember

The component should receive meaningful data such as active, status or loading. It should translate that data into a visual state. This keeps the UI reusable.

MODULE 2 • PART 8

Project 1: Responsive Profile Cards

Goal

Build a responsive profile-card section using reusable components. First implement it with pure CSS, then rebuild the same UI with Tailwind.

Directory

```
src/
■■■ components/
■   ■■■ ProfileCard.jsx
■   ■■■ ProfileList.jsx
■■■ App.jsx
■■■ App.css
■■■ index.css
```

Step 1: data

App.jsx data

```
const users = [
  {
    id: 1,
    name: "Riya",
    role: "Frontend Developer",
    location: "Kaithal",
    active: true
  },
  {
    id: 2,
    name: "Aman",
    role: "Backend Developer",
    location: "Delhi",
    active: false
  },
  {
    id: 3,
    name: "Neha",
    role: "UI Developer",
    location: "Chandigarh",
    active: true
  }
]
```

Step 2: component

ProfileCard.jsx

```
function ProfileCard({ user }) {
  return (
    <article className="profile-card">
      <div className="profile-card__avatar">
        {user.name[0]}
      </div>

      <h2>{user.name}</h2>
      <p>{user.role}</p>
      <small>{user.location}</small>

      <span className={
        user.active
          ? "status status--online"
          : "status status--offline"
      }>
        {user.active ? "Active" : "Offline"}
      </span>
    </article>
  )
}

export default ProfileCard
```

Project 1: Pure CSS layout

App.css

```
.profile-grid {
  display: grid;
  grid-template-columns:
    repeat(auto-fit, minmax(220px, 1fr));
  gap: 20px;
}

.profile-card {
  padding: 24px;
  border-radius: 18px;
  background: white;
  border: 1px solid #e2e8f0;
  box-shadow: 0 10px 30px rgba(15,23,42,.08);
}

.profile-card__avatar {
  width: 52px;
  height: 52px;
  display: grid;
  place-items: center;
  border-radius: 50%;
  background: #dbeafe;
  color: #1d4ed8;
  font-weight: 800;
  font-size: 1.25rem;
```

App.css (continued)

```
}
```

Project 1: Tailwind version

Tailwind ProfileCard

```
function ProfileCard({ user }) {
  return (
    <article className="
      rounded-2xl border border-slate-200
      bg-white p-6 shadow-lg
    ">
      <div className="
        grid h-13 w-13 place-items-center
        rounded-full bg-blue-100
        text-xl font-extrabold text-blue-700
      ">
        {user.name[0]}
      </div>

      <h2 className="mt-4 text-xl font-bold">
        {user.name}
      </h2>

      <p className="text-slate-600">{user.role}</p>
      <small className="text-slate-400">{user.location}</small>

      <span className={`
        mt-4 inline-block rounded-full px-3 py-1
        text-sm font-bold
        ${user.active
```

Tailwind ProfileCard (continued)

```
          ? "bg-green-100 text-green-700"
          : "bg-slate-100 text-slate-600"}
      `}>
        {user.active ? "Active" : "Offline"}
      </span>
    </article>
  )
}
```

Project 1: List component

ProfileList.jsx

```
import ProfileCard from "./ProfileCard"

function ProfileList({ users }) {
  return (
    <section className="profile-grid">
      {users.map((user) => (
        <ProfileCard
          key={user.id}
          user={user}
        />
      ))}
    </section>
  )
}

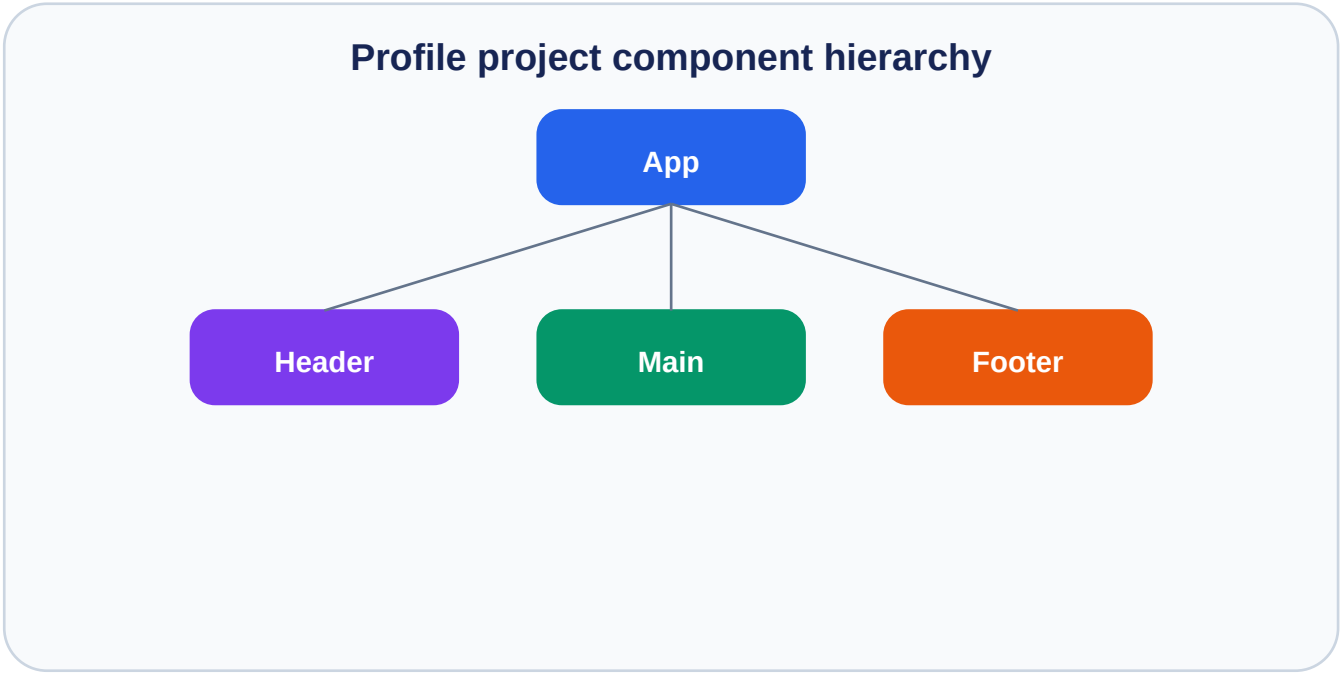
export default ProfileList
```

App composition

```
function App() {
  return (
    <main className="app">
      <header>
        <h1>Team</h1>
        <p>Our developers</p>
      </header>

      <ProfileList users={users} />
    </main>
  )
}
```

Expected component tree



MODULE 2 • PART 9

Project 2: Registration Form

Goal

Build a registration form using reusable Input and Button components. Implement the visual design with pure CSS first, then recreate the same form using Tailwind.

Directory

Directory

```
src/
■■■ components/
■   ■■■ Input.jsx
■   ■■■ Button.jsx
■   ■■■ RegisterForm.jsx
■■■ App.jsx
■■■ App.css
■■■ index.css
```

RegisterForm

RegisterForm.jsx

```
import { useState } from "react"
import Input from "./Input"
import Button from "./Button"

function RegisterForm() {
  const [form, setForm] = useState({
    name: "",
    email: "",
    password: ""
  })

  function handleChange(event) {
    const { name, value } = event.target

    setForm((current) => ({
      ...current,
      [name]: value
    }))
  }

  function handleSubmit(event) {
    event.preventDefault()

    if (!form.name || !form.email || !form.password) {
      alert("All fields are required")
```

RegisterForm.jsx (continued)

```
      return
    }

    alert(`Welcome, ${form.name}!`)
  }

  return (
    <form onSubmit={handleSubmit} className="form">
      <Input
        label="Name"
        name="name"
        value={form.name}
        onChange={handleChange}
      />

      <Input
        label="Email"
        name="email"
        type="email"
        value={form.email}
        onChange={handleChange}
      />

      <Input
        label="Password"
```

RegisterForm.jsx (continued)

```
        name="password"
        type="password"
        value={form.password}
        onChange={handleChange}
      />

      <Button type="submit">
        Create account
      </Button>
    </form>
  )
}

export default RegisterForm
```

Project 2: Tailwind form version

Tailwind Input

```
function Input({
  label,
  name,
  type = "text",
  value,
  onChange
}) {
  return (
    <label className="grid gap-2">
      <span className="font-bold text-slate-700">
        {label}
      </span>

      <input
        name={name}
        type={type}
        value={value}
        onChange={onChange}
        className="
          rounded-lg border border-slate-300
          px-3 py-2 outline-none
          focus:border-blue-600
          focus:ring-4 focus:ring-blue-100
        "
      />
```

Tailwind Input (continued)

```
    </label>
  )
}

export default Input
```

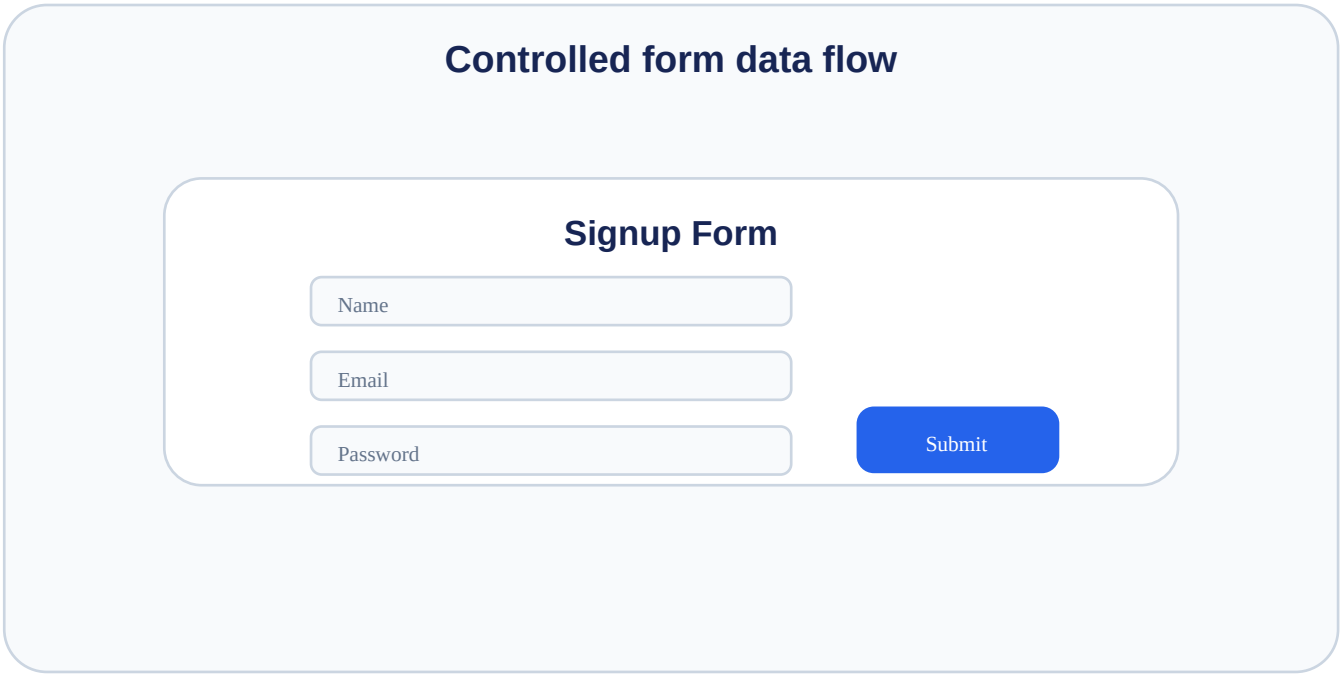
Tailwind Button

```
function Button({ children, type = "button" }) {
  return (
    <button
      type={type}
      className="
        rounded-lg bg-blue-600
        px-4 py-2 font-bold text-white
        transition hover:bg-blue-700
        focus:outline-none focus:ring-4
        focus:ring-blue-200
      "
    >
      {children}
    </button>
  )
}

export default Button
```

Form flow

Controlled form data flow



Module 2 Practical Exercises

Exercise 1: Styling Challenge

Level: Intermediate

- Create a reusable ProductCard component.
- Make a pure CSS version with hover effects.
- Make a Tailwind version with the same visual structure.
- Accept title, price, category and featured as props.
- Show a Featured badge only when featured is true.

Exercise 2: Responsive Dashboard

Level: Intermediate+

- Create a dashboard with Header, Sidebar and MainContent components.
- Use Flexbox for the top-level layout.
- Use CSS Grid for dashboard cards.
- Make the dashboard one column on mobile and multiple columns on larger screens.
- Implement it once with pure CSS and once with Tailwind.

Exercise 3: Event Playground

Level: Intermediate

- Create buttons for Click, Double Click, Mouse Enter and Mouse Leave.
- Print a message to the console from each handler.
- Create one handler that reads an input's value.
- Demonstrate the difference between onClick={handleClick} and onClick={handleClick()}.

Exercise 4: Registration Form

Level: Intermediate+

- Build name, email and password inputs.
- Keep all fields controlled.
- Prevent the default submit behavior.
- Validate that no field is empty.
- Show a success message after valid submission.

Exercise 5: Reusable UI Kit

Level: Advanced

- Create Button, Badge, Card and Input components.
- Each component must accept useful props.
- Create at least three Button variants.
- Create success, warning and danger Badge variants.
- Use the components to build a small profile page.

Exercise Solutions

Solution 1: ProductCard

Solution 1: Tailwind ProductCard

```
function ProductCard({
  title,
  price,
  category,
  featured
}) {
  return (
    <article className="
      rounded-2xl border border-slate-200
      bg-white p-6 shadow-lg
      transition hover:-translate-y-1
    ">
      <small className="text-slate-500">
        {category}
      </small>

      <h2 className="my-2 text-xl font-bold">
        {title}
      </h2>

      <strong className="text-lg text-blue-600">
        ■{price}
      </strong>

      {featured && (
```

Solution 1: Tailwind ProductCard (continued)

```
        <span className="
          ml-2 rounded-full bg-yellow-100
          px-2 py-1 text-xs font-bold
          text-yellow-700
        ">
          Featured
        </span>
      )}
    </article>
  )
}

export default ProductCard
```

Solution 1: pure CSS core

```
.product-card {
  padding: 24px;
  border: 1px solid #e2e8f0;
  border-radius: 16px;
  background: white;
  box-shadow: 0 10px 25px rgba(15,23,42,.08);
  transition: transform .2s ease;
}

.product-card:hover {
  transform: translateY(-4px);
}
```

Solution 2: Responsive Dashboard

Solution 2: JSX

```
function Dashboard() {
  return (
    <div className="dashboard">
      <aside className="sidebar">
        <h2>Admin</h2>
        <a href="#">Dashboard</a>
        <a href="#">Users</a>
        <a href="#">Courses</a>
      </aside>

      <main className="dashboard__main">
        <h1>Dashboard</h1>

        <section className="dashboard__grid">
          <div className="stat">Users: 120</div>
          <div className="stat">Courses: 18</div>
          <div className="stat">Revenue: ■42k</div>
          <div className="stat">Orders: 86</div>
        </section>
      </main>
    </div>
  )
}
```

Solution 2: pure CSS

```
.dashboard {
  min-height: 100vh;
  display: flex;
}

.sidebar {
  width: 220px;
  padding: 24px;
  background: #172554;
  color: white;
}

.sidebar a {
  display: block;
  margin-top: 12px;
  color: white;
}

.dashboard__main {
  flex: 1;
  padding: 24px;
}

.dashboard__grid {
  display: grid;
```

Solution 2: pure CSS (continued)

```
  grid-template-columns:
    repeat(2, minmax(0, 1fr));
  gap: 16px;
}

.stat {
  padding: 24px;
  border-radius: 14px;
  background: white;
  border: 1px solid #e2e8f0;
}

@media (max-width: 700px) {
  .dashboard {
    display: block;
  }

  .sidebar {
    width: auto;
  }

  .dashboard__grid {
    grid-template-columns: 1fr;
  }
}
```

Solution 2: Tailwind

```
<div className="
  min-h-screen
  md:flex
">
  <aside className="
    bg-blue-950 p-6 text-white
    md:w-56
  ">
    <h2 className="text-xl font-bold">Admin</h2>
    <nav className="mt-4 grid gap-2">
      <a href="#">Dashboard</a>
      <a href="#">Users</a>
      <a href="#">Courses</a>
    </nav>
  </aside>

  <main className="flex-1 p-6">
    <h1 className="text-3xl font-bold">
      Dashboard
    </h1>

    <section className="
      mt-6 grid gap-4
      sm:grid-cols-2
      lg:grid-cols-4
```

Solution 2: Tailwind (continued)

```
    ">
      <div className="rounded-xl bg-white p-5 shadow">Users: 120</div>
      <div className="rounded-xl bg-white p-5 shadow">Courses: 18</div>
      <div className="rounded-xl bg-white p-5 shadow">Revenue: ■42k</div>
      <div className="rounded-xl bg-white p-5 shadow">Orders: 86</div>
    </section>
  </main>
</div>
```

Solution 3: Event Playground

Solution 3

```
function EventPlayground() {
  function handleClick() {
    console.log("Clicked")
  }

  function handleDoubleClick() {
    console.log("Double clicked")
  }

  function handleMouseEnter() {
    console.log("Mouse entered")
  }

  function handleMouseLeave() {
    console.log("Mouse left")
  }

  function handleInput(event) {
    console.log(event.target.value)
  }

  return (
    <section>
      <button onClick={handleClick}>
        Click
```

Solution 3 (continued)

```
      </button>

      <button onDoubleClick={handleDoubleClick}>
        Double Click
      </button>

      <div
        onMouseEnter={handleMouseEnter}
        onMouseLeave={handleMouseLeave}
      >
        Hover area
      </div>

      <input onChange={handleInput} />
    </section>
  )
}
```

Debugging test

Temporarily write console.log() inside handlers and use the browser console. If nothing appears, check whether the handler is actually being passed to the element.

Solution 4: Registration Form

Solution 4

```
import { useState } from "react"

function RegistrationForm() {
  const [form, setForm] = useState({
    name: "",
    email: "",
    password: ""
  })

  const [message, setMessage] = useState("")

  function handleChange(event) {
    const { name, value } = event.target

    setForm((current) => ({
      ...current,
      [name]: value
    }))
  }

  function handleSubmit(event) {
    event.preventDefault()

    if (
      !form.name ||
```

Solution 4 (continued)

```
      !form.email ||
      !form.password
    ) {
      setMessage("Please fill every field.")
      return
    }

    setMessage("Registration successful!")
  }

  return (
    <form onSubmit={handleSubmit}>
      <input
        name="name"
        value={form.name}
        onChange={handleChange}
      />

      <input
        name="email"
        type="email"
        value={form.email}
        onChange={handleChange}
      />
```

Solution 4 (continued)

```
      <input
        name="password"
        type="password"
        value={form.password}
        onChange={handleChange}
      />

      <button type="submit">
        Register
      </button>

      {message && <p>{message}</p>}
    </form>
  )
}
```

Solution 5: Mini UI Kit

Solution 5

```
function Card({ title, children }) {
  return (
    <section className="
      rounded-2xl border border-slate-200
      bg-white p-6 shadow-lg
    ">
      <h2 className="mb-3 text-xl font-bold">
        {title}
      </h2>

      {children}
    </section>
  )
}

function Badge({ type = "success", children }) {
  const styles = {
    success: "bg-green-100 text-green-700",
    warning: "bg-yellow-100 text-yellow-700",
    danger: "bg-red-100 text-red-700"
  }

  return (
    <span className={`
      rounded-full px-3 py-1
```

Solution 5 (continued)

```
      text-sm font-bold
      ${styles[type]}
    `}>
      {children}
    </span>
  )
}

function App() {
  return (
    <main className="grid gap-5 p-6">
      <Card title="Profile">
        <p className="mb-3 text-slate-600">
          Frontend developer
        </p>
        <Badge type="success">
          Active
        </Badge>
      </Card>

      <Card title="Warning">
        <Badge type="warning">
          Pending
        </Badge>
      </Card>
```

Solution 5 (continued)

```
      <Card title="Error">
        <Badge type="danger">
          Failed
        </Badge>
      </Card>
    </main>
  )
}
```

Module 2 Revision Sheet

Key concepts to remember

- React components can receive both data props and event-handler props.
- CSS class names are applied with className.
- Inline styles use JavaScript object syntax and camelCase property names.
- CSS files are useful for hover, focus, media queries and complex reusable styling.
- Tailwind uses utility classes directly in JSX.
- Tailwind v4's Vite setup uses @tailwindcss/vite and @import "tailwindcss". ■cite■turn0search1■
- Flexbox is usually best for one-dimensional layouts.
- Grid is usually best for two-dimensional card/page layouts.
- Responsive design should adapt layout to available space.
- Event handlers are passed as functions, not called during render. ■cite■turn0search0■
- preventDefault() prevents default browser behavior.
- stopPropagation() prevents an event from reaching parent handlers.
- Controlled inputs get their value from React state.
- Reusable components should expose meaningful variation through props.

Quick questions

1. When would you prefer a CSS file over inline styling?
2. What is the difference between Flexbox and Grid?
3. Why is onClick={handleClick} different from onClick={handleClick()}?
4. What does event.preventDefault() do?
5. What does event.stopPropagation() do?
6. What is a controlled input?
7. Why should a reusable Button accept an onClick prop?
8. How can a component change its appearance based on a status prop?
9. What is the current Tailwind Vite plugin package?
10. Why should you avoid copying old Tailwind v3 setup instructions into a v4 project?

Module 2 Final Practical Assessment

Build a **Course Management Dashboard**. The project should look like a real small application, not five divs wearing different colors.

Requirements

- Use Vite + React.
- Create Header, Sidebar, CourseCard, CourseList, Badge, Button and CourseForm components.
- Use an array of course objects.
- Render courses with map() and stable keys.
- Use props for course data and event handlers.
- Use a responsive Grid for course cards.
- Use Flexbox for navigation/sidebar areas.
- Build the CourseForm with controlled inputs.
- Prevent default form submission.
- Validate required fields.
- Implement both a pure CSS version and a Tailwind version of at least the CourseCard and layout.
- Use dynamic status styling for Active, Draft and Closed courses.
- Keep the project organized in a components directory.

Target directory

Final project structure

```
course-dashboard/
■■■ src/
■   ■■■ components/
■   ■   ■■■ Header.jsx
■   ■   ■■■ Sidebar.jsx
■   ■   ■■■ CourseCard.jsx
■   ■   ■■■ CourseList.jsx
■   ■   ■■■ CourseForm.jsx
■   ■   ■■■ Badge.jsx
■   ■   ■■■ Button.jsx
■   ■■■ App.jsx
■   ■■■ App.css
■   ■■■ index.css
■   ■■■ main.jsx
■■■ package.json
■■■ vite.config.js
```

Assessment checklist

- No console errors.
- No missing-key warnings.
- All buttons work.
- Form does not reload the page.
- Empty fields are handled.
- Responsive layout works on mobile and desktop.
- At least one component is implemented with pure CSS and Tailwind alternatives.
- Students can explain the data flow and event flow.

Instructor Teaching Plan

Part	Topic	Time	Focus
Part 1	Styling	45–60 min	Show the same card in inline, CSS and Tailwind.
Part 2	Tailwind	45–60 min	Install current v4 workflow and build a card.
Part 3	Layouts	60 min	Build navbar + responsive card grid.
Part 4	Events	45–60 min	Use buttons, inputs and event objects.
Part 5	Forms	60–90 min	Build controlled registration form.
Part 6	Reusable UI	60 min	Build Button, Badge, Input and Card.
Part 7	Dynamic Styling	45 min	Drive visual states from props.
Part 8	Project	90–120 min	Profile cards and dashboard.
Part 9	Assessment	90–120 min	Course Management Dashboard.

Teaching sequence

For each concept: explain → type a tiny example → break it intentionally → fix it → build a reusable version → give an exercise. This makes the student understand why the pattern exists instead of memorizing syntax.

Module 2 complete

At the end of this module, students should be able to take a plain React component and turn it into a responsive, styled, interactive UI. They should also be able to read both pure CSS and Tailwind implementations and understand the trade-offs between them.

Next module: React state and interactivity in depth, including useState and state-driven UI.